

Classroom B - C Lab Worksheet (Single File)

Student Task

Build **one single C file** named `week11_lab_b.c`.

Rules:

1. Use only one `#include <stdio.h>`.
2. Put each question in a separate function.
3. Write only one `main()` and call all question functions inside `main()` in order.
4. Questions 1-5 are normal coding tasks.
5. Questions 6-10 start from intentionally broken code. In each of those questions, fix the broken function before testing it.
6. Everything needed to solve the worksheet is described here. Do not assume any extra starter code.
7. You may use loops and pointer arithmetic freely.

Implementation notes:

- You may reuse helper functions across questions.
- Define helper functions before the question functions that call them, or write function prototypes before `main`.
- Do not hard-code the expected output. Your output must come from your functions.
- For functions that return a yes/no result, return `1` for yes/true and `0` for no/false.

Use this struct definition at the top of your file (after the `#include`):

```
typedef struct{
    char name[ 30 ];
    double price;
    int quantity;
} Item;
```

Total: 100 points

Each question is worth **10 points**.

Suggested `main()`:

```
int main(void) {
    example1_item_basics();
    example2_inventory_report();
    example3_price_operations();
    example4_search_and_filter();
    example5_sort_by_price();
    example6_broken_init_item();
    example7_broken_total_value();
    example8_broken_cheapest();
}
```

```
example9_broken_restock();
example10_broken_remove_item();
return 0;
}
```

Question 1 - Item Basics [10 points]

Write functions to print and manually initialize an `Item` struct.

Requirements:

1. Write `void print_item(Item *p)`.
 - Print the item in this format: `name: $price x quantity`
 - Print price with two decimal places.
 - Example: `Pencil: $1.50 x 200`
2. Write `void print_inventory(Item *first, Item *last)`.
 - Print each item in the range `[first, last)` using `print_item`.
3. Write `void copy_name(char *dest, const char *src)`.
 - Copy characters from `src` to `dest` one by one until `'\0'`. Write the terminating `'\0'` too.
4. Inside `example1_item_basics`, create and print:

```
Item a = { "Pencil", 1.50, 200 };
Item b;
copy_name( b.name, "Eraser" );
b.price = 0.75;
b.quantity = 350;
```

5. Print both items using `print_item`.

Goal:

- Practice struct initialization, member access with `->`, and manual string copying into a char array.

Expected output:

```
--- Q1: Item Basics ---
Pencil: $1.50 x 200
Eraser: $0.75 x 350
```

Question 2 - Inventory Report [10 points]

Generate a summary report from an array of items.

Requirements:

1. Write `double item_value(Item *p)`.
 - Return `p->price * p->quantity`.
2. Write `double total_value(Item *first, Item *last)`.
 - Return the sum of `item_value` for all items in the range.
3. Write `int total_units(Item *first, Item *last)`.
 - Return the total quantity of all items combined.
4. Inside `example2_inventory_report`, use:

```
Item stock[4] = {  
    { "Ruler",      3.25,  80 },  
    { "Glue",       2.50, 150 },  
    { "Scissors",   7.90,  45 },  
    { "Tape",       1.80, 200 }  
};
```

5. Print inventory, total value, and total units.

Expected output:

```
--- Q2: Inventory Report ---  
Ruler: $3.25 x 80  
Glue: $2.50 x 150  
Scissors: $7.90 x 45  
Tape: $1.80 x 200  
Total value: $1,350.50  
Total units: 475
```

Hint:

- For the total value formatting with comma, you can print it as `printf( "Total value: $%.2f\n", total )` — the comma in the expected output above is just for readability. Your output without comma (`$1375.50`) will also be accepted.

Question 3 - Price Operations [10 points]

Modify prices across an entire inventory.

Requirements:

1. Write `void apply_discount(Item *first, Item *last, double percent)`.
 - Reduce the price of every item by the given percentage.
 - `p->price *= (1.0 - percent / 100.0);`
 2. Write `void apply_tax(Item *first, Item *last, double percent)`.
 - Increase the price of every item by the given percentage.
 - `p->price *= (1.0 + percent / 100.0);`
 3. Inside `example3_price_operations`, use:
-

```
Item stock[3] = {
    { "Marker",    4.00, 100 },
    { "Notepad",   10.00,  60 },
    { "Stapler",   15.00,  30 }
};
```

4. Print inventory, apply 20% discount, print again, then apply 10% tax, and print the final result.

Expected output:

```
--- Q3: Price Operations ---
Before:
Marker: $4.00 x 100
Notepad: $10.00 x 60
Stapler: $15.00 x 30
After 20% discount:
Marker: $3.20 x 100
Notepad: $8.00 x 60
Stapler: $12.00 x 30
After 10% tax:
Marker: $3.52 x 100
Notepad: $8.80 x 60
Stapler: $13.20 x 30
```

Question 4 - Search and Filter [10 points]

Search and filter items based on criteria.

Requirements:

1. Write `Item *find_by_name(Item *first, Item *last, char *target)`.
 - Return a pointer to the first item whose name matches `target`, or return `0` (NULL) if not found.
 - Compare strings character by character: both strings are equal if every character matches and both end at `'\0'` at the same position.
2. Write `void print_expensive(Item *first, Item *last, double min_price)`.
 - Print all items with `price >= min_price`.
3. Inside `example4_search_and_filter`, use:

```
Item stock[5] = {
    { "Pen",        2.50, 300 },
    { "Notebook",   12.00,  75 },
    { "Folder",     5.50, 120 },
    { "Binder",     8.75,  50 },
    { "Clips",      1.20, 500 }
};
```

Expected output:

```
--- Q4: Search and Filter ---
Found: Notebook: $12.00 x 75
Crayon not found.
Items with price >= $5.00:
Notebook: $12.00 x 75
Folder: $5.50 x 120
Binder: $8.75 x 50
```

Hint:

- To compare two strings, walk both pointers together. If all characters match and both strings end at `'\0'`, they are equal.

Question 5 - Sort by Price [10 points]

Sort an array of items by price in ascending order.

Requirements:

1. Write `void sort_by_price_asc(Item *first, Item *last)`.
 - Use selection sort: in each pass, find the item with the minimum price in the unsorted portion and swap it to the front.
 - Swap entire `Item` structs using a `Item temp` variable.
2. Inside `example5_sort_by_price`, use:

```
Item stock[5] = {
    { "Binder",    8.75,  50 },
    { "Clips",     1.20, 500 },
    { "Pen",       2.50, 300 },
    { "Notebook", 12.00,  75 },
    { "Folder",    5.50, 120 }
};
```

3. Sort and print the inventory.

Expected output:

```
--- Q5: Sort by Price ---
Clips: $1.20 x 500
Pen: $2.50 x 300
Folder: $5.50 x 120
Binder: $8.75 x 50
Notebook: $12.00 x 75
```

Question 6 - Broken Code: init_item [10 points]

The function below is supposed to initialize an item. It has **one bug** that causes the name to be garbage or empty.

```
void init_item( Item *p, char *name, double price, int qty ){
    p->price = price;
    p->quantity = qty;
}
```

Fix `init_item` and test it inside `example6_broken_init_item` with:

```
Item x;
init_item( &x, "Highlighter", 3.25, 80 );
print_item( &x );
```

Expected output after fix:

```
--- Q6: Broken init_item ---
Highlighter: $3.25 x 80
```

Hint:

- The function sets price and quantity, but is it doing anything with the name?

Question 7 - Broken Code: total_value_broken [10 points]

The function below is supposed to calculate the total inventory value (sum of `price * quantity` for each item). It has **one bug** that gives a result that is far too small.

```
double total_value_broken( Item *first, Item *last ){
    double total = 0;
    Item *p;
    for( p = first; p < last; p++ )
        total += p->price;
    return total;
}
```

Fix `total_value_broken` and test it inside `example7_broken_total_value` with:

```
Item stock[2] = {
    { "Book", 25.00, 10 },
```

```

        { "Lamp", 40.00, 5 }
    };
    printf( "total value: %.2f\n", total_value_broken( stock, stock + 2 ) );

```

Expected output after fix:

```

--- Q7: Broken total_value ---
total value: $450.00

```

Hint:

- The value of one item is not just its price. What else should be multiplied?

Question 8 - Broken Code: cheapest_broken [10 points]

The function below is supposed to return a pointer to the item with the lowest price. It has **one bug** that actually finds the most expensive item.

```

Item *cheapest_broken( Item *first, Item *last ){
    Item *best = first, *p;
    for( p = first + 1; p < last; p++ )
        if( p->price > best->price ) best = p;
    return best;
}

```

Fix `cheapest_broken` and test it inside `example8_broken_cheapest` with:

```

Item stock[3] = {
    { "Brush", 6.50, 40 },
    { "Canvas", 18.00, 20 },
    { "Paint", 4.25, 90 }
};
Item *c = cheapest_broken( stock, stock + 3 );
printf( "cheapest: %s (%.2f)\n", c->name, c->price );

```

Expected output after fix:

```

--- Q8: Broken cheapest ---
cheapest: Paint ($4.25)

```

Hint:

- To find the smallest, should you look for items with a price greater or less than the current best?

Question 9 - Broken Code: restock_broken [10 points]

The function below is supposed to add **amount** units to every item in the inventory. It has **one bug** that replaces the quantity instead of adding to it.

```
void restock_broken( Item *first, Item *last, int amount ){
    Item *p;
    for( p = first; p < last; p++ )
        p->quantity = amount;
}
```

Fix **restock_broken** and test it inside **example9_broken_restock** with:

```
Item stock[2] = {
    { "Ink",    5.00, 30 },
    { "Paper", 10.00, 50 }
};
printf( "before:\n" );
print_inventory( stock, stock + 2 );
restock_broken( stock, stock + 2, 20 );
printf( "after restock +20:\n" );
print_inventory( stock, stock + 2 );
```

Expected output after fix:

```
--- Q9: Broken restock ---
before:
Ink: $5.00 x 30
Paper: $10.00 x 50
after restock +20:
Ink: $5.00 x 50
Paper: $10.00 x 70
```

Hint:

- Should the new quantity be just **amount**, or should it include the original quantity?

Question 10 - Broken Code: remove_item_broken [10 points]

The function below is supposed to remove an item at index **idx** from the inventory by shifting all items after it one position to the left. The function should return the new size. It has **one bug** that shifts in the wrong direction, corrupting the array.


```
int remove_item_broken( Item items[], int size, int idx ){
    int i;
    for( i = idx; i < size - 1; i++ )
        items[ i + 1 ] = items[ i ];
    return size - 1;
}
```

Fix `remove_item_broken` and test it inside `example10_broken_remove_item` with:

```
Item stock[4] = {
    { "A", 1.00, 10 },
    { "B", 2.00, 20 },
    { "C", 3.00, 30 },
    { "D", 4.00, 40 }
};
int size = 4;
printf( "before:\n" );
print_inventory( stock, stock + size );
size = remove_item_broken( stock, size, 1 );
printf( "after removing index 1:\n" );
print_inventory( stock, stock + size );
```

Expected output after fix:

```
--- Q10: Broken remove_item ---
before:
A: $1.00 x 10
B: $2.00 x 20
C: $3.00 x 30
D: $4.00 x 40
after removing index 1:
A: $1.00 x 10
C: $3.00 x 30
D: $4.00 x 40
```

Hint:

- To shift elements left, each position should receive the value from the position to its right, not from the position to its left.